Pensamento Computacional
INF05008

Aula 3 – Expressões e Funções

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

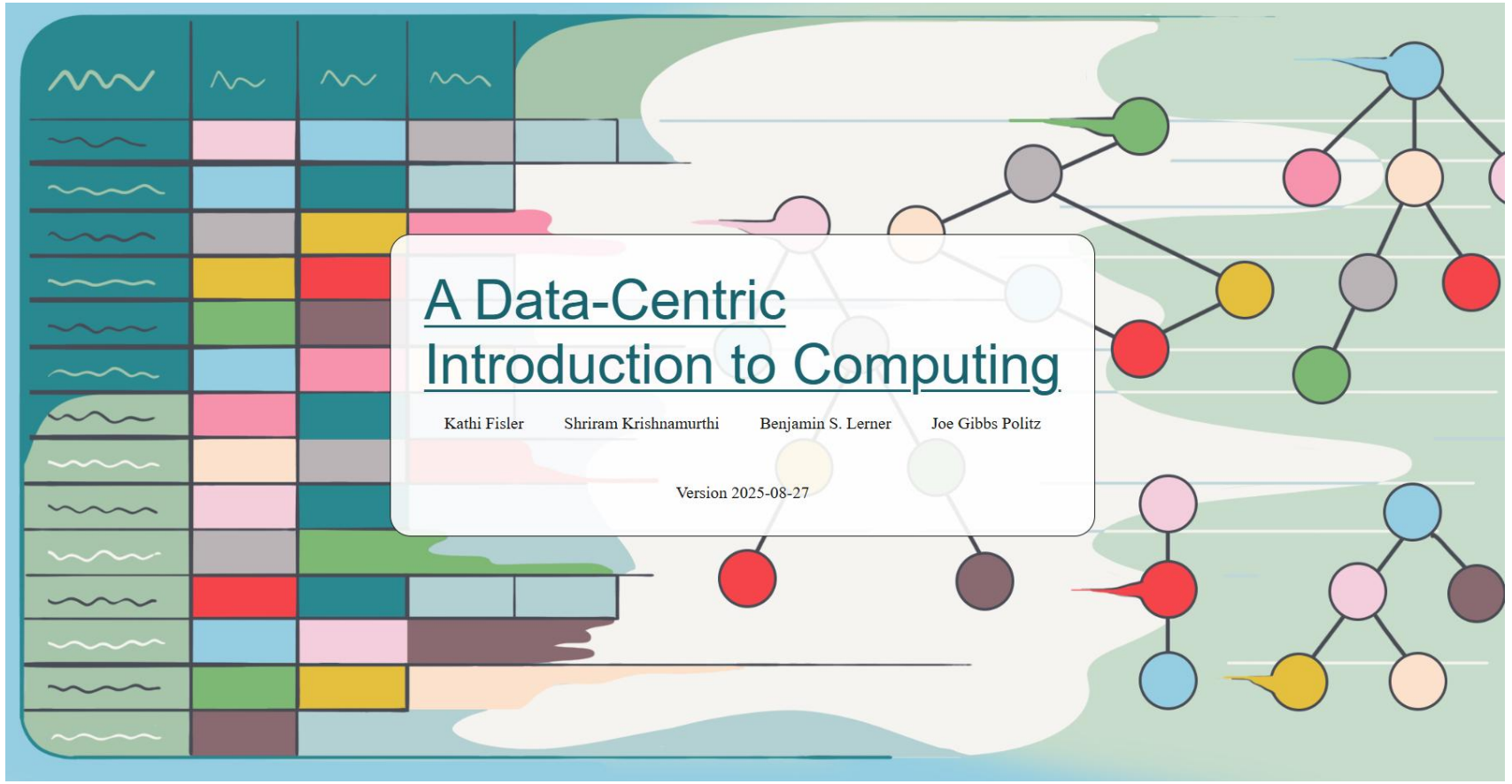
Instituto de Informática

Departamento de Informática Teórica

Slides baseados no material da Prof. Leila Ribeiro.

Bibliografia

- Livro Online: https://dcic-world.org

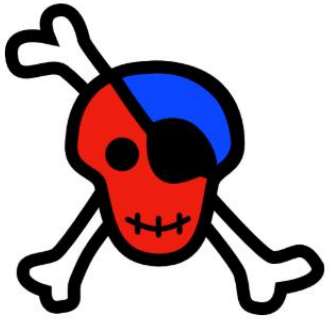


Ambientes de Programação

Pyret Interactive Editor
for VSCode



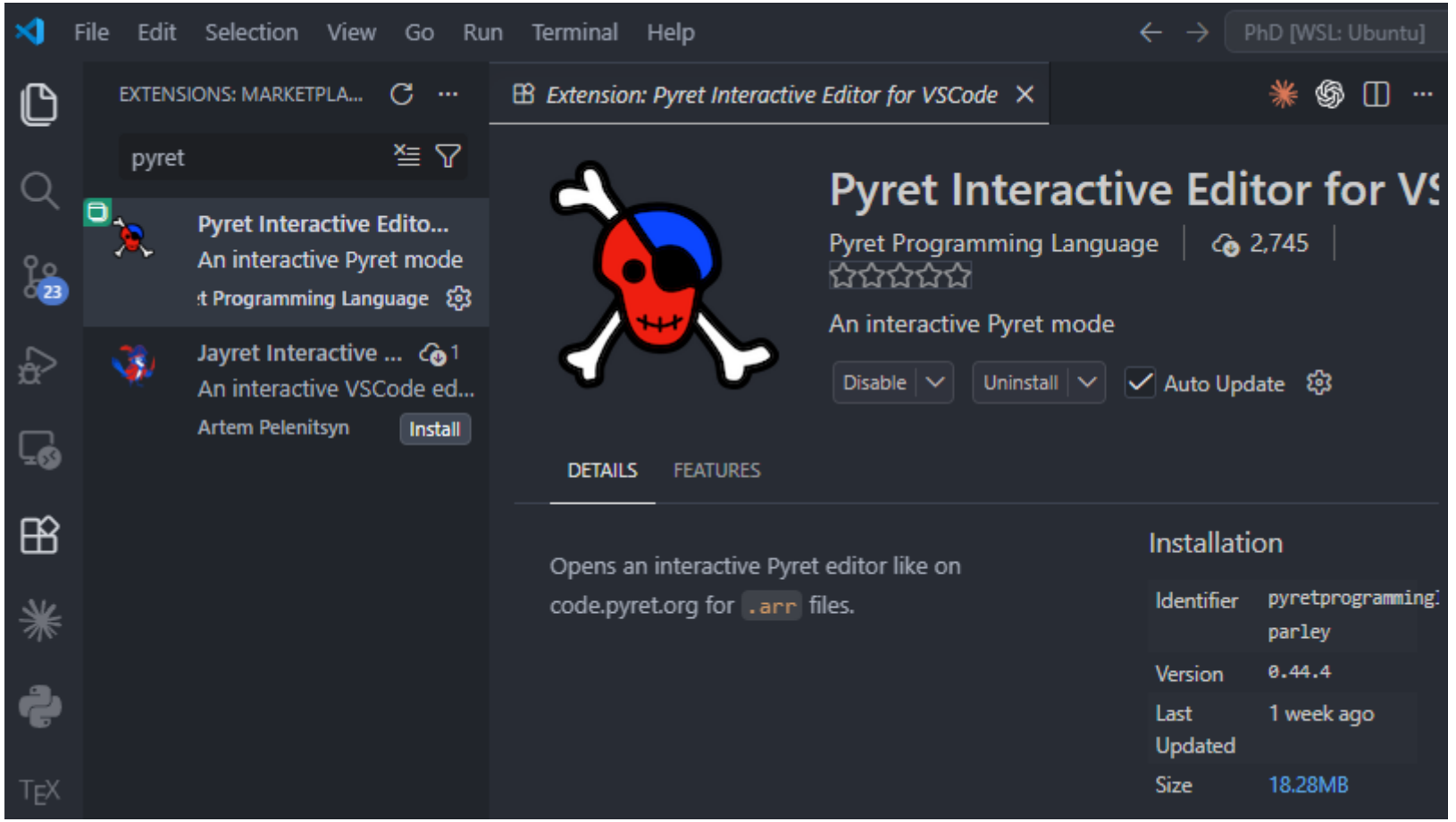
https://code.pyret.org



Open Editor
to start coding immediately

Sign in
to save and view programs

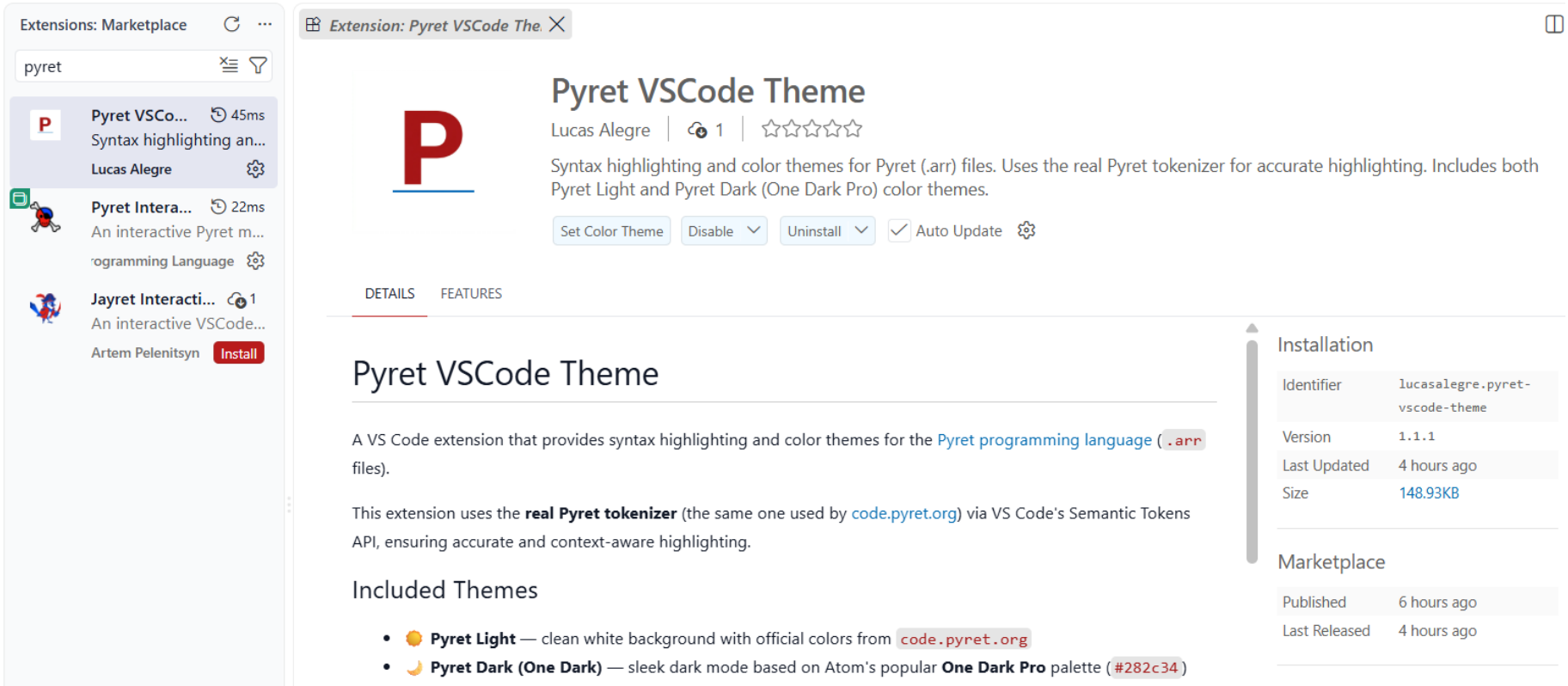
VSCode – Pyret Extension



- Instale a extensão do Pyret no seu VSCode.

Sugestão Opcional – **Pyret VSCode Theme**

- Extensão com dois temas de cores (claro e escuro) para usar no VSCode



Linguagem de Programação: Pyret

- Documentação da Linguagem: https://pyret.org/docs/latest/index.html

- Por quê Pyret? - https://pyret.org/pyret-code/
  - Criada para ser uma introdução a computação
  - Inspirada em outras linguagens funcionais (Scheme, Racket)
  - Sintaxe simples próxima da sintaxe de Python
  - De Pyret a Python:

  https://dcic-world.org/2025-08-27/booklet_pyret-to-python.html

- Extensão sufixo ".arr"
  - http://dialectblog.com/2011/05/24/pirate-accent/

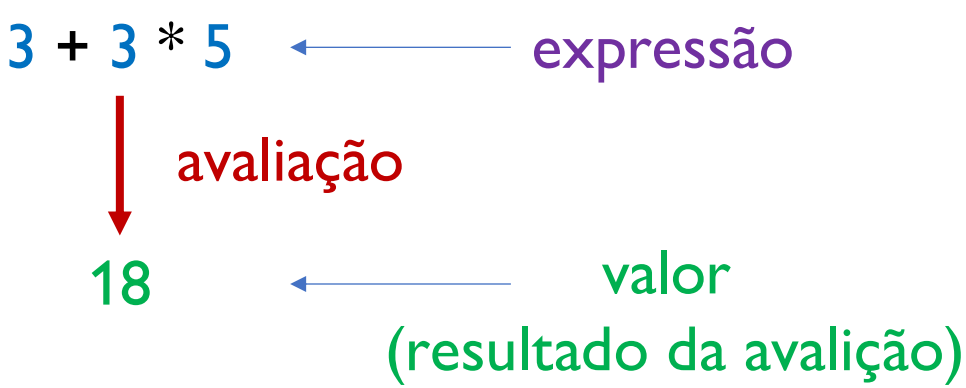


Expressão

P. O que é uma expressão?

Definição: Uma expressão é uma descrição sintática de um valor, que é determinado a partir da avaliação da expressão.

Exemplo:

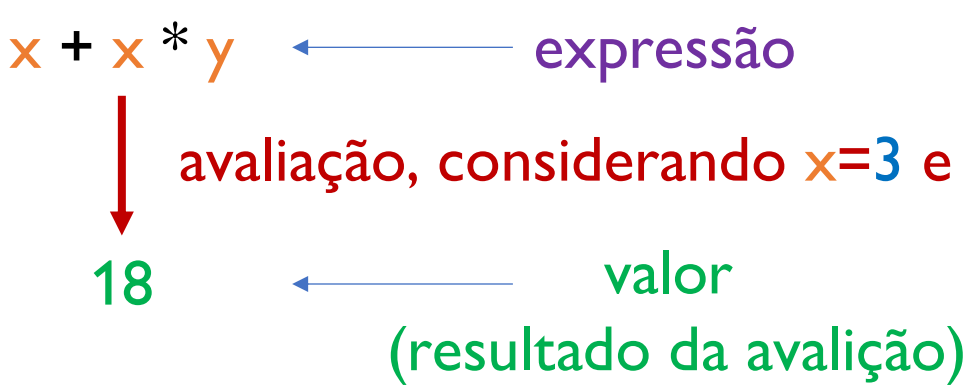


Expressão

P. O que é uma expressão?

Definição: Uma expressão é uma descrição sintática de um valor, que é determinado a partir da avaliação da expressão.

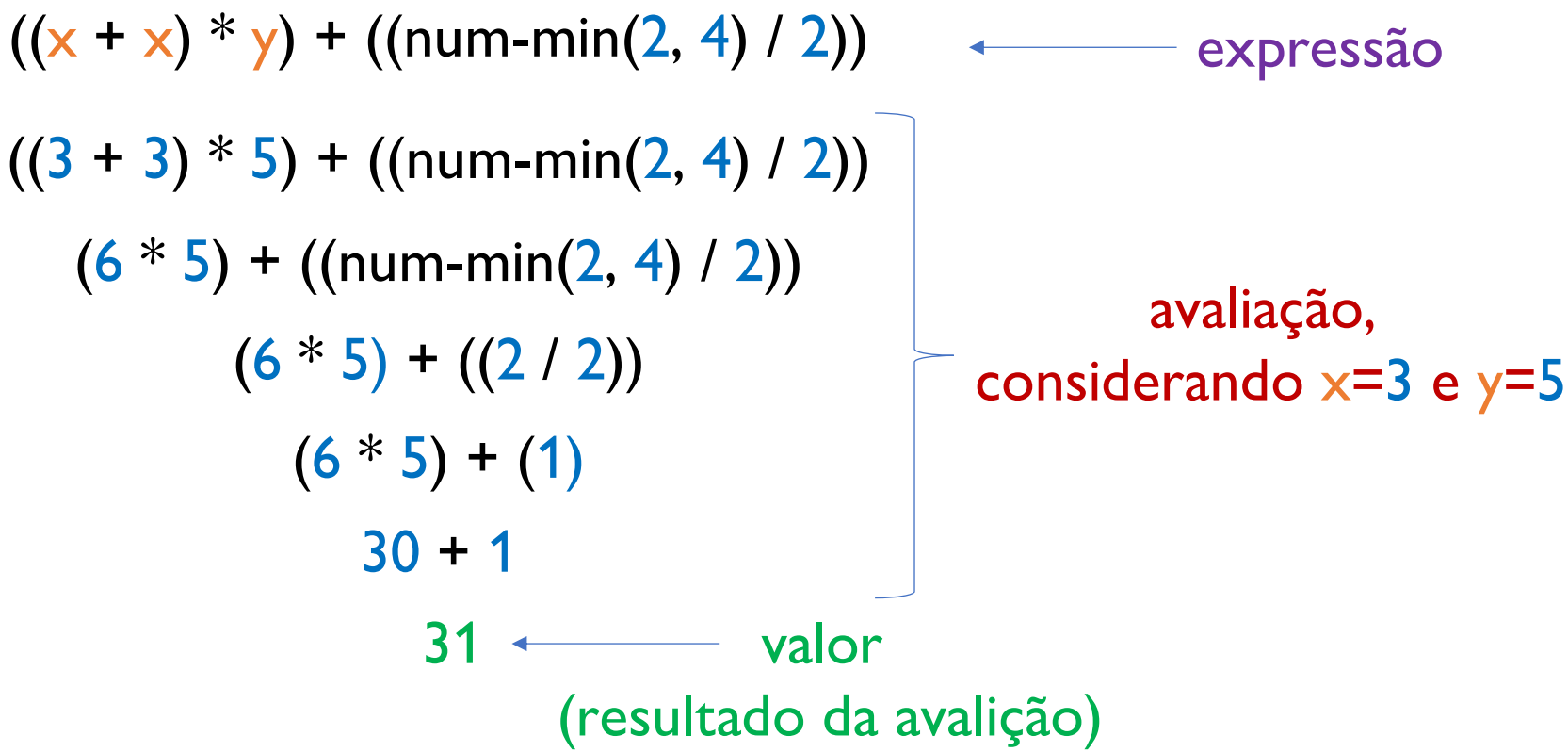
Exemplo:



Expressões podem conter variáveis, cujo valor precisa ser determinado para permitir a avaliação da expressão.

Expressão

- Expressões podem ser tão complicadas quanto quisermos:



Expressões em Pyret

- Em Pyret, devemos indicar a ordem de precedência das operações com parênteses para evitar ambiguidade:

Erro:

Reading this expression errored:

definitions://:2:0-2:9

```
3 3 + 3 * 5
```

The + and * operations are at the same grouping level. Add parentheses to group the operations, and make the order of operations clear.

Correto:

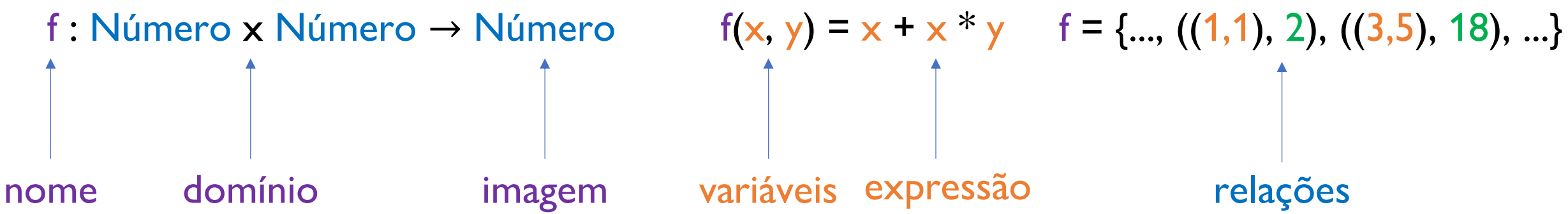
```
››› 3 + (3 * 5)
18
```

- Também devemos sempre colocar espaços entre os operadores:
  - 3+4 vs. 3 + 4

Funções

P. O que é uma função?

Definição: Uma função é uma relação entre valores (de entrada e saída).



- O domínio representa o conjunto de possíveis entradas.
- A imagem representa o conjunto de possíveis saídas.

Aplicação de Função

- Toda função é descrita por um expressão (tipicamente composta de múltiplas expressões) que avalia para um único valor.

f : Número x Número → Número

f(x, y) = x + x * y

variáveis expressão

Definição

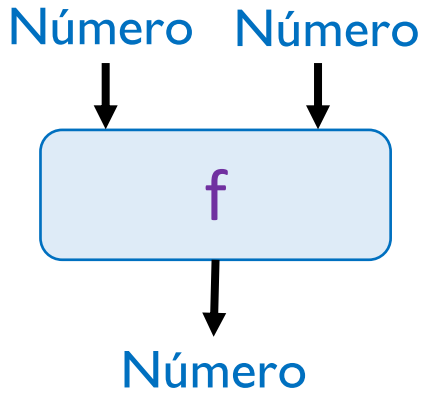
f(3, 5) = 3 + 3 * 5 = 18

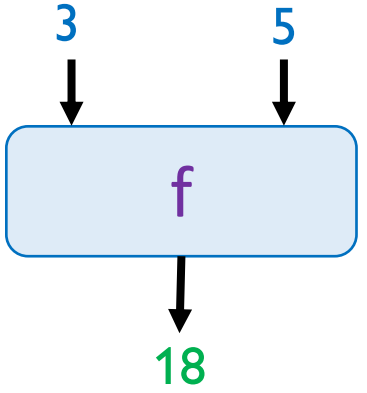
valores

substituição das variáveis por valores

Aplicação

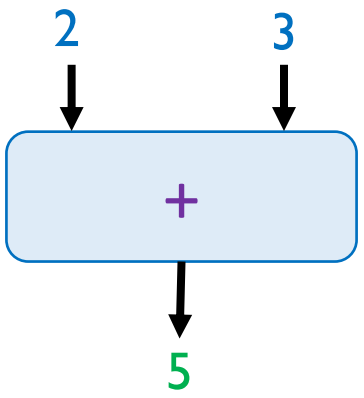
resultado/valor da avaliação da expressão



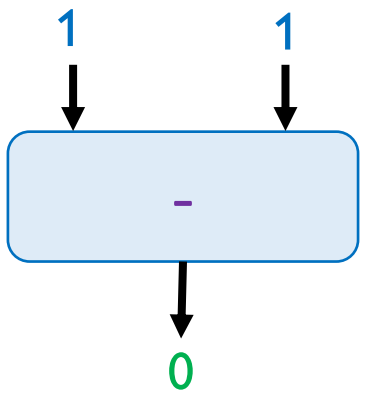


Aplicação de Função: Notação Visual

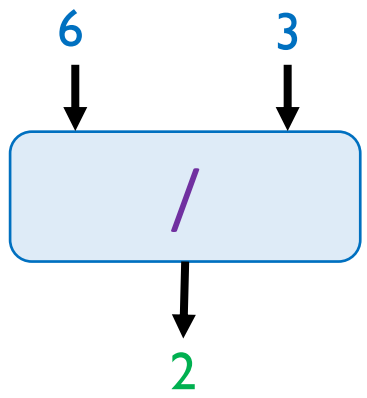
2 + 3



1 - 1



6 / 3



- Operações aritméticas (+, -, *, /) também podem ser vistas como funções.

Strings

- E se quisermos representar informação não numérica, por exemplo, o nome de alguém? Para isso, usamos *strings*.
- Exemplo:
  - "Lucas"
  - "Instituto de Informática"
  - "No meio do caminho havia uma pedra."
  - "aaaaAAAAAAAAAAAAAAaaaAAAAAAAAAAAAaaaaaag iopaej4309u3094"

- Strings são *case-sensitive, i.e.,* "oi" ≠ "Oi"

Imagens

Pyret tem suporte ao tipo de dados Image.

- Exemplo:

```
››› circle(30, "solid", "red")
```



```
››› circle(30, "outline", "blue")
```

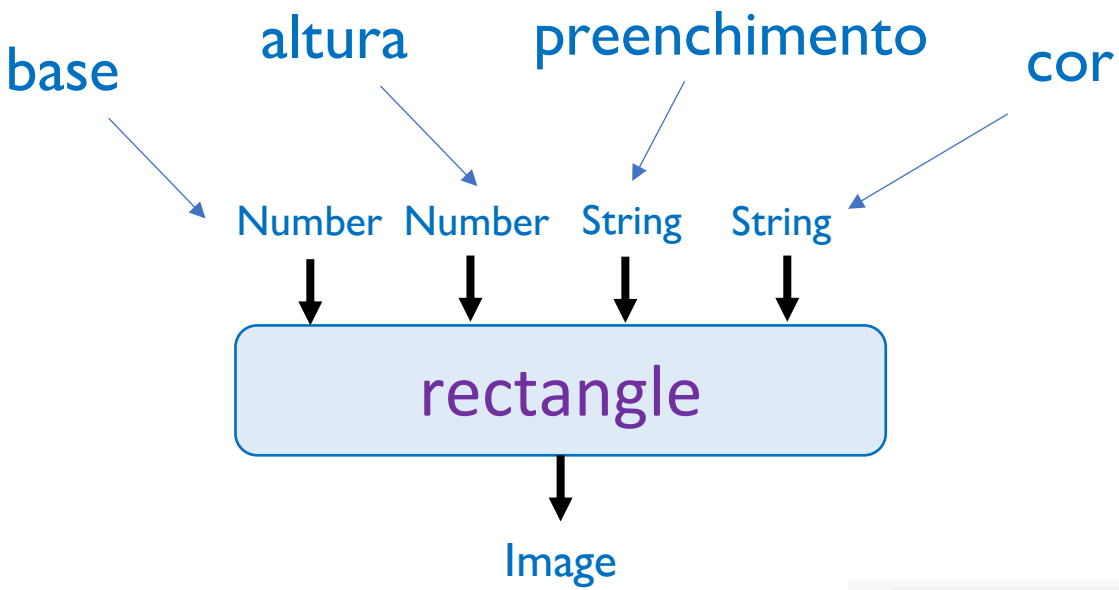


```
››› rectangle(100, 50, "solid", "purple")
```



Funções

Podemos criar funções que utilizam vários tipos de dados.

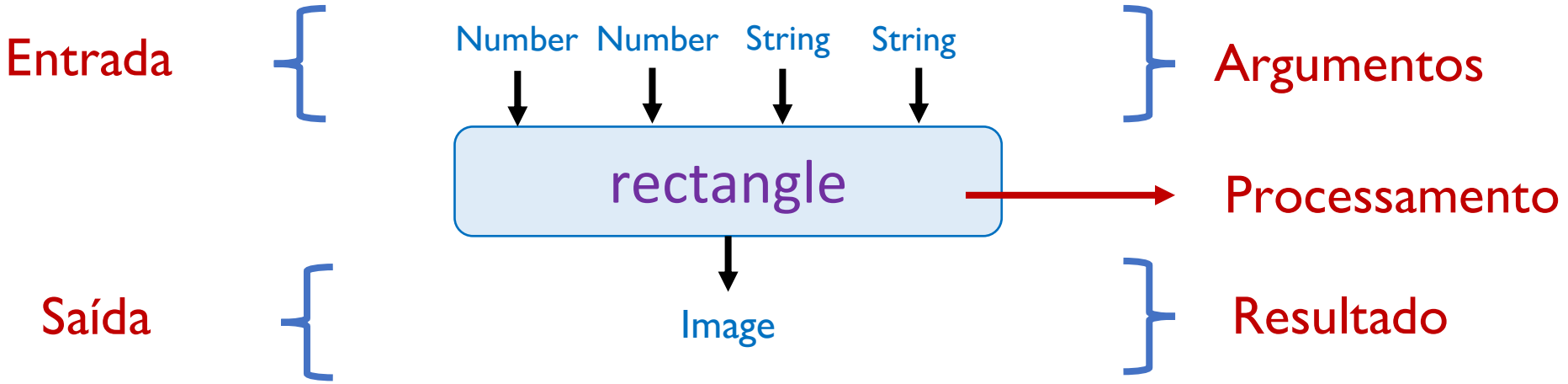


O preenchimento pode ser "solid" ou "outline".

```
>>> rectangle(100, 50, "solid", "purple")
```

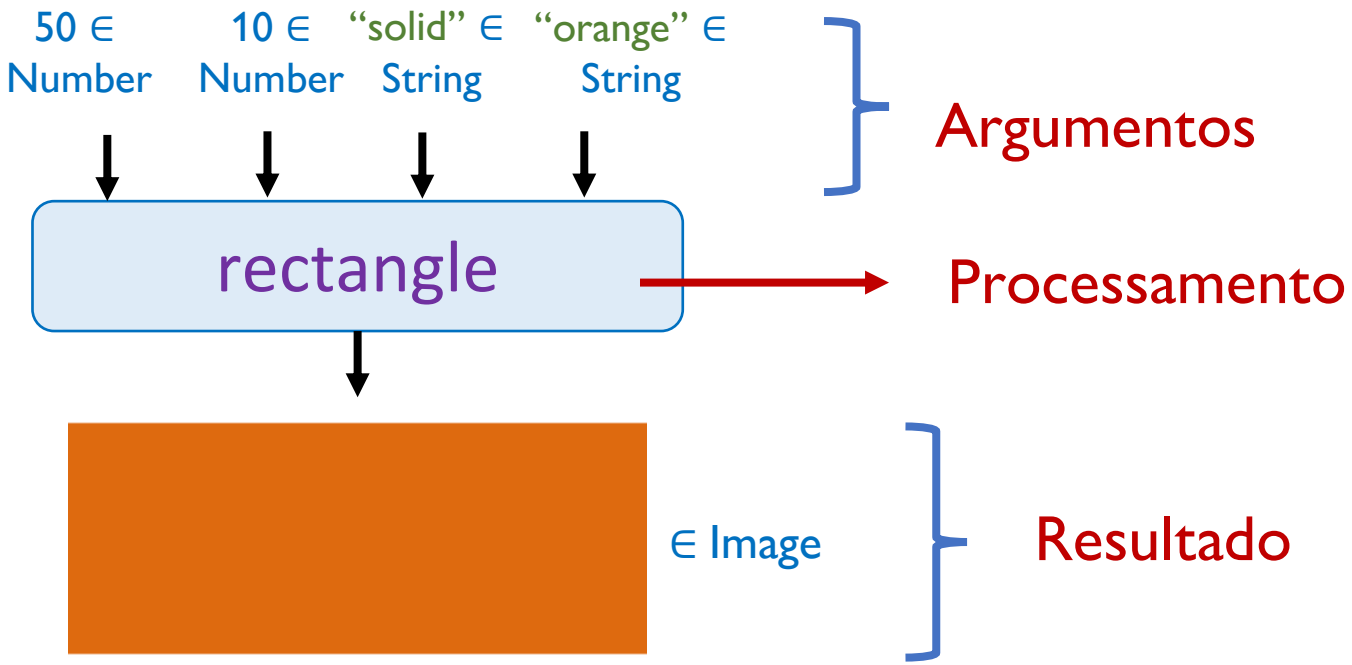


Funções

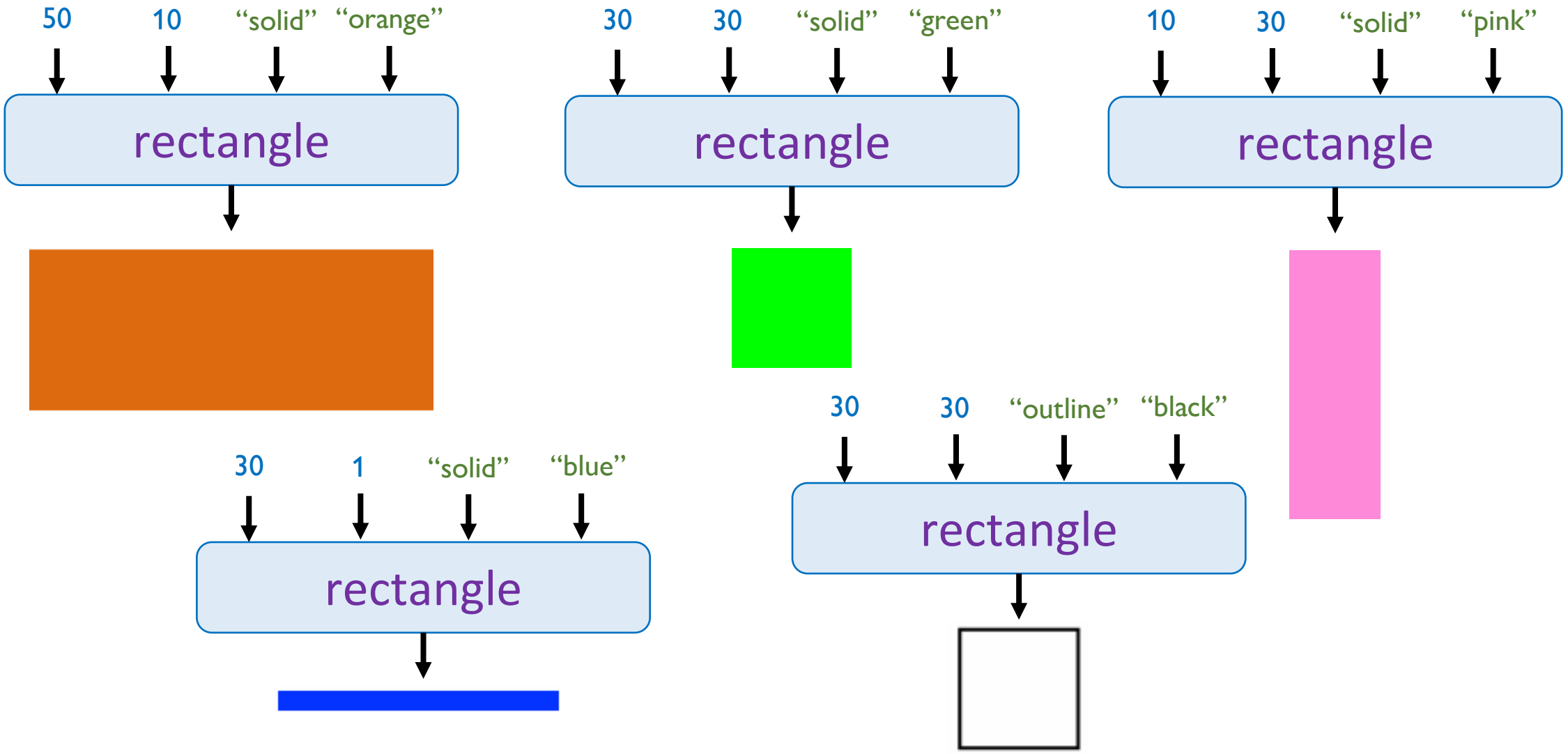


- Uma função é **genérica**: pode ser usada para desenhar qualquer retângulo.

Funções

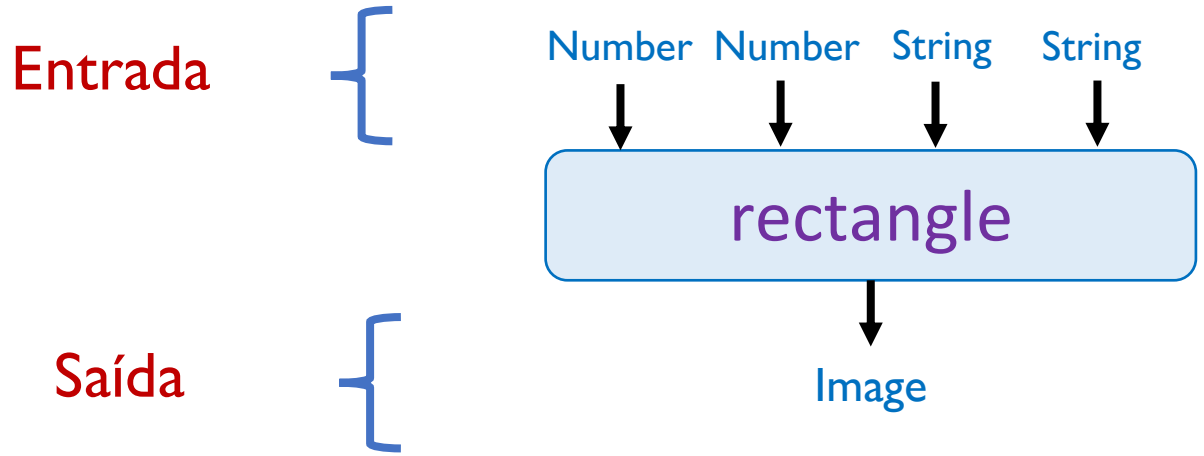

50 ∈ Number
10 ∈ Number
"solid" ∈ String
"orange" ∈ String
Argumentos
rectangle
Processamento
∈ Image
Resultado

Aplicação da Função



Tipos

P. Por que devemos explicitar os tipos das entradas e saídas?



- Para saber o que precisa para desenhar o retângulo
- Para saber a ordem das entradas

Principais Tipos

Number

1

0

1001

154

-5

...

7.2

3.141516

String

"Oi"

"AAAAA"

"Era uma vez..."

"é os guri"

"kkkkkkkkk"

Boolean

true

false

Image



Desenhando Figuras

- Em Pyret, temos disponível várias funções para desenhar figuras

- Ver: https://pyret.org/docs/horizon/image.html

- Vamos desenhar algumas figuras:
- https://code.pyret.org/editor#share=1ekiVDT3Vv72NoeYEfmhGWj6Lg0KCNUuU&v=918dd1f

code.pyret.org



code.pyret.org



Open Editor

to start coding immediately

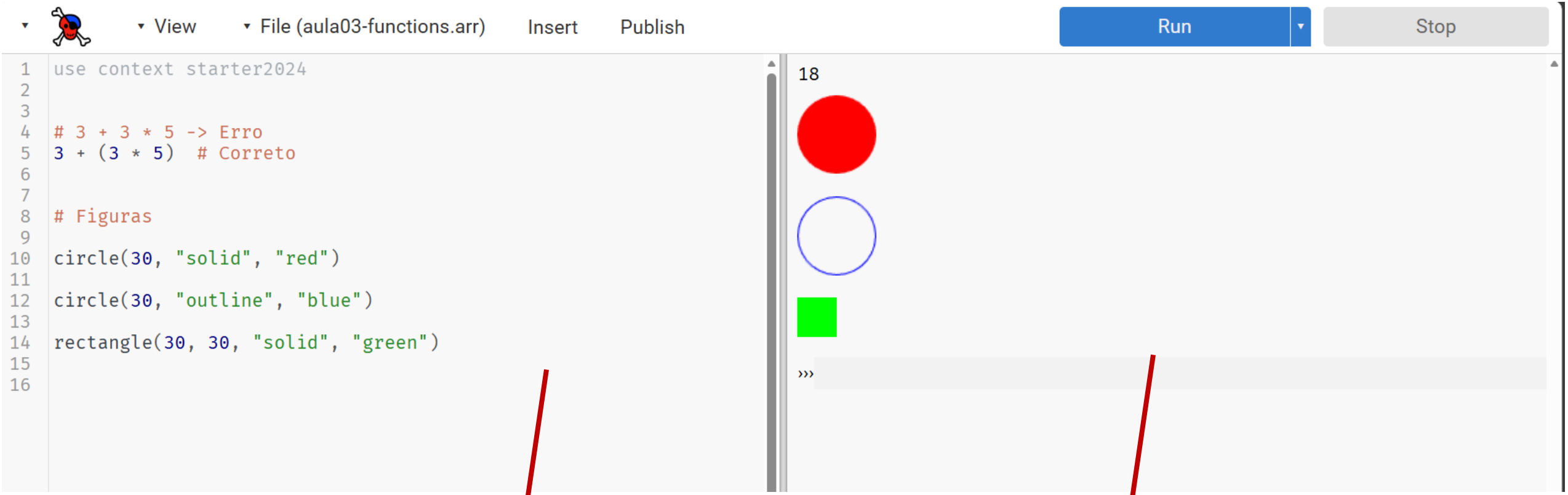
Sign in

to save and view programs

You can also check out our book that uses Pyret or our curricula.

Programa e Prompt

Definição. Um programa é uma sequência de expressões que queremos executar.



Programa /
Janela de Definições

Janela de Interações /
Prompt

Cores

Cores pré-definidas em Pyret:

- https://pyret.org/docs/horizon/color.html#(part._s~3acolor-constants)



3.1.15.2 Predefined colors

The following colors are predefined constants:

- orange: abc123 abc123
- red: abc123 abc123
- orange-red: abc123 abc123
- tomato: abc123 abc123
- dark-red: abc123 abc123
- fire-brick: abc123 abc123
- crimson: abc123 abc123
- deep-pink: abc123 abc123
- maroon: abc123 abc123
- green: abc123 abc123
- lawn-green: abc123 abc123
- chartreuse: abc123 abc123
- green-yellow: abc123 abc123
- yellow-green: abc123 abc123
- medium-forest-green: abc123 abc123
- olive-drab: abc123 abc123
- dark-olive-green: abc123 abc123
- dark-sea-green: abc123 abc123
- lime: abc123 abc123
- dark-green: abc123 abc123
- lime-green: abc123 abc123
- forest-green: abc123 abc123
- royal-blue: abc123 abc123
- dodger-blue: abc123 abc123
- deep-sky-blue: abc123 abc123
- cornflower-blue: abc123 abc123
- steel-blue: abc123 abc123
- light-sky-blue: abc123 abc123
- dark-turquoise: abc123 abc123
- cyan: abc123 abc123
- aqua: abc123 abc123
- dark-cyan: abc123 abc123
- teal: abc123 abc123
- sky-blue: abc123 abc123
- cadet-blue: abc123 abc123

...

Exercício

Exercício. Construa essas bandeiras:



Podemos dar nomes para expressões:

```
barra_branca = rectangle(30, 60, "solid", "white")
barra_azul = rectangle(30, 60, "solid", "blue")
barra_vermelha = rectangle(30, 60, "solid", "red")
```

E depois usá-los:

```
beside(beside(barra_azul, barra_branca), barra_vermelha)
```

Exercício

Exercício. Construa essas bandeiras:



Podemos dar nomes também para as bandeiras!

```
bandeira_franca = beside(beside(barra_azul, barra_branca),
barra_vermelha)
bandeira_italia = beside(beside(barra_verde, barra_branca),
barra_vermelha)
bandeira_belgica = beside(beside(barra_preta, barra_amarela),
barra_vermelha)
bandeira_nigeria = beside(beside(barra_verde, barra_branca),
barra_verde)
```

Quando damos um **nome** para algo, estamos trabalhando a **Abstração**!

Abstração

P. Quais as vantagens de definir os nomes barra_verde, barra_branca, ... ?

- Fica mais simples:
  - Melhor legibilidade
  - Código mais compacto
  - Mais fácil de entender

```
bandeira_italia =
  beside(
    beside(
      rectangle(30, 60, "solid", "darkgreen"),
      rectangle(30, 60, "solid", "white")),
    rectangle(30, 60, "solid", "red"))

bandeira_italia =
  beside(
    beside(
      barra_verde,
      barra_branca),
    barra_vermelha)
```

- Podemos reutilizar um mesmo nome várias vezes:

```
bandeira_franca = beside(beside(barra_azul, barra_branca), barra_vermelha)
```

Abstração

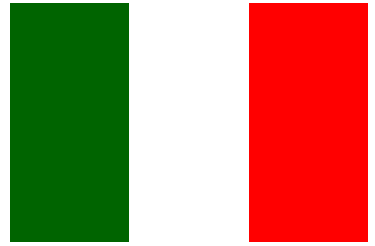
P. Quais as vantagens de definir os nomes barra_verde, barra_branca, ... ?

- Podemos mudar mais facilmente:

```
barra_vermelha = rectangle(30, 60, "solid", "red")
barra_verde = rectangle(30, 60, "solid", "darkgreen")
```



```
bandeira_italia =
  beside(
    beside(
      barra_verde,
      barra_branca),
    barra_vermelha)
```



Podemos fazer melhor?

- O que tem em comum nessas definições?
- O que tem de diferente nessas definições?

```
barra_branca = rectangle(30, 60, "solid", "white")
barra_azul = rectangle(30, 60, "solid", "blue")
barra_vermelha = rectangle(30, 60, "solid", "red")
barra_verde = rectangle(30, 60, "solid", "darkgreen")
barra_preta = rectangle(30, 60, "solid", "black")
barra_amarela = rectangle(30, 60, "solid", "gold")
```

Desenhando uma Barra

P. Imagine que você quer pedir para alguém desenhar uma barra para você de uma cor X. Como você pediria?

- "Desenhe um retângulo de 30 por 60 todo pintado da cor X."

P. Agora imagine que a pessoa já sabe que uma barra é um retângulo, sabe o tamanho e que ela deve ser totalmente pintada. Como você pediria?

- "Desenha uma barra da cor X."

Desenhando uma Barra

P. E se a pessoa não souber como desenhar a barra, como você explicaria como ela deve fazer esse desenho?

- Quando eu disser **barra** e uma cor você deve desenhar um retângulo de 30 por 60 todo pintado nesta cor.

- Explicação em "computês" (linguagem Pyret):

```
fun barra(cor):
  rectangle(30, 60, "solid", cor)
end
```

Estamos descrevendo uma ação **genérica**: funciona para qualquer cor!

barra é uma Função

```
fun barra(cor):
  rectangle(30, 60, "solid", cor)
end
```

Definição

Valor

```
››› barra("blue")
```



Resultado da avaliação da expressão

```
››› barra("yellow")
```


Aplicação

O uso de **variáveis** permite trabalhar a **Generalização**!

Como construir uma função

Passo 1: Escolher um nome para a função

Passo 2: Definir os tipos das entradas e saída e dar um nome aos argumentos

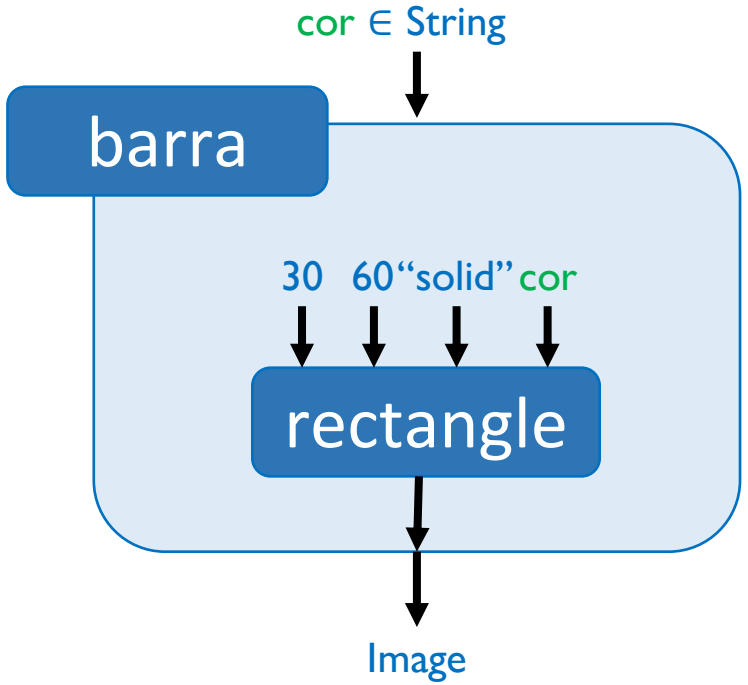
Passo 3: Definir o que a função faz com a entrada para gerar a saída

Criar uma Barra

Dado o nome de uma cor, cria uma barra sólida de 30x60 com esta cor.

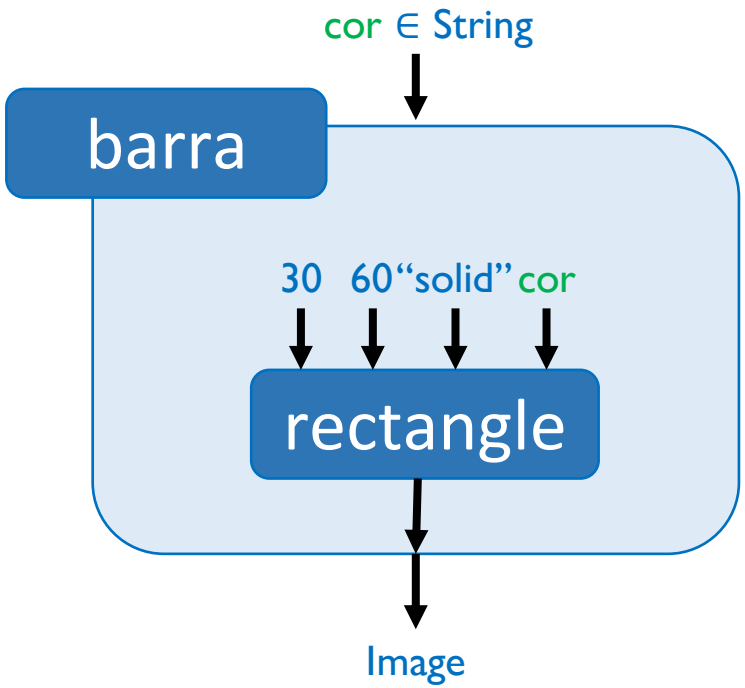
Entrada: Um nome de cor (string).

Saída: Uma imagem de barra solida de 30x60 na cor dada.



Como construir uma função

```
fun barra(cor :: String) -> Image:
  rectangle(30, 60, "solid", cor)
end
```



barra é uma função que,
dada uma cor, desenha um
retângulo de tamanho
30 x 60 desta cor.

Como construir uma função

- Sintaxe em Pyret:

```
fun <nome-da-função>(<parametros>):
    <expressão>
end
```

- A expressão da função também é chamada de corpo da função.
- Toda função possui uma única expressão
  - Tipicamente composta de múltiplas expressões/funções

Exercícios

Exercício 1: Construa expressões para desenhar essas bandeiras:







Funções úteis (ver https://pyret.org/docs/horizon/image.html)

```
overlay-align :: (
    place-x :: XPlace,
    place-y :: YPlace,
    img1 :: Image,
    img2 :: Image
)
-> Image
```

```
star :: (
    side-length :: Number,
    mode :: String,
    color :: ImageColor
)
-> Image
```

```
beside :: (
    img1 :: Image,
    img2 :: Image
)
-> Image
```

```
above :: (
    img1 :: Image,
    img2 :: Image
)
-> Image
```

Exercícios

Exercício 2: Construa uma função que, dadas 3 cores, desenha uma bandeira nas cores dadas:

bandeira("red", "gold", "black") 

Exercício 3: Construa uma função para desenhar bandeiras no seguinte formato:





Exercício 4: Construa uma função que, dado um número, mostra o número dentro de um círculo vermelho.

Exercício 5: Uma padaria vende pão por 2 reais, bolinho por 3 reais e croissant por 4 reais. Construa uma função que, dado o números de pães, bolinhos e croissants que o cliente quer comprar, calcula o que ele deve pagar, e mostra dentro de um círculo vermelho.

Funções Úteis (Exercício 3, 4 e 5)

```
text :: (
   string :: String,
   font-size :: Number,
   color :: ImageColor
)
-> Image
```

Constructs an image of `string`, using the given font size and color.

Examples:

```
››› text("Hello", 24, "olive")
```

Hello

```
››› text("Goodbye", 36, "indigo")
```

```
num-to-string :: (n :: Number) -> String
```

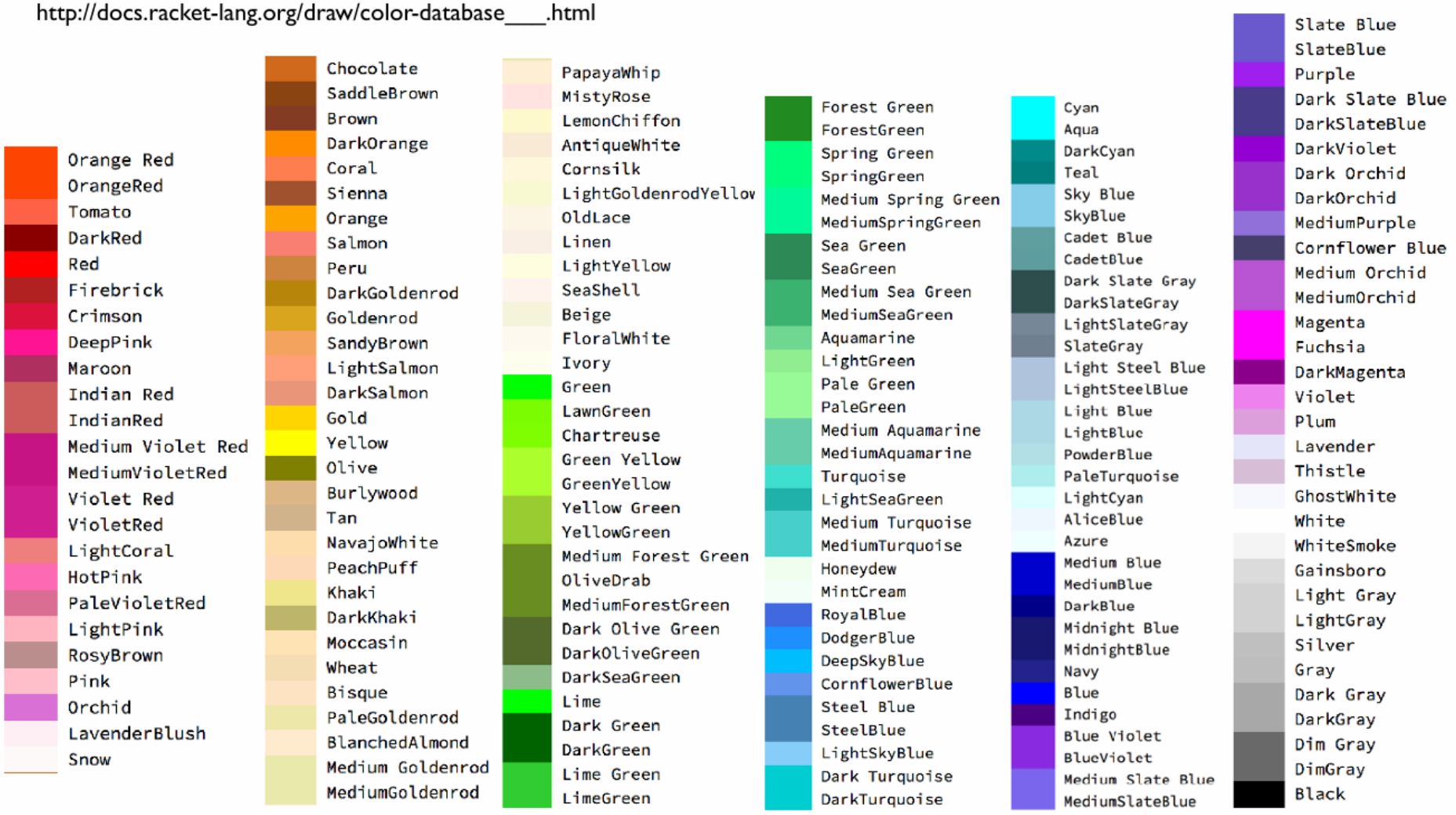
Returns a `String` representing a literal form of the number.

Examples:

```
check:
  num-to-string(2.5) is "5/2"
  num-to-string(2) is "2"
  num-to-string(2/3) is "2/3"
  num-to-string(~2.718) is "~2.718"
  num-to-string(~6.022e23) is "~6.022e+23"
end
```

Cores

http://docs.racket-lang.org/draw/color-database___.html



Linguagem:
Sintaxe e Semântica

Linguagem

- O objetivo de uma linguagem é transmitir informação.

- Para se construir uma linguagem são necessários:
  - Símbolos
  - Sintaxe
  - Semântica

Símbolos

- Quando escrevemos um texto, usamos os símbolos permitidos pela linguagem que estamos utilizando…

Sintaxe

- Para escrever as frases, seguimos as regras da sintaxe da linguagem, que dizem como devem ser formadas as sentenças.

- Em linguagem natural, as frases tendem a ter muitas palavras (não se concentrando estritamente na essência a ser transmitida).

- Linguagens de programação tendem a ser muito mais objetivas.

Semântica

- Semântica é o significado das palavras ou símbolos de uma linguagem.



- Novos símbolos podem ser criados, e sua semântica definida:
  - "Sepá"
  - Internet
  - MESA = circle(100, "solid", "lightgrey")

Execução de Programas

- Para entender um texto, precisamos conhecer a semântica de todas palavras usadas no texto.

- Para executar um programa, o computador precisa entender a semântica de todas palavras usadas no programa.

P. Como o computador faz isso?

- O computador guarda uma tabela com as palavras conhecidas e seu significado (como um dicionário).
- Essa lista pode ser modificada durante a execução de um programa…

Tabela de Nomes

Nome	Tipo	Valor (significado)
false	constante	Valor lógico “falso”
num-sqr	função	Função que eleva um número ao quadrado
circle	função	Função que gera a imagem de um círculo

- Em Pyret, estes nomes são pré-definidos, sua semântica está embutida no processador da linguagem.
- Mas podemos definir novos nomes!

Definindo Nomes

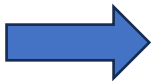
- Novos nomes podem ser criados, e sua semântica definida:

```
PI = 3.14
MESA = circle(200, "solid", "lightgray")
fun media(a :: Number, b :: Number) -> Number:
  doc: "Dado dois números, retorna a média entre eles."
  (a + b) / 2
end
```

Nome	Tipo	Valor (significado)
PI	constante	3.14
MESA	constante	Imagem de um círculo cinza de raio 200

media	função	`fun media(a, b):`
		`  (a + b) / 2`
		`end`

Exemplo



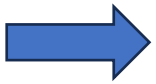
```
fun media(a :: Number, b :: Number) -> Number:
  (a + b) / 2
end

media(2, 4)
```

Nome	Tipo	Valor (significado)

media	função	`fun media(a, b):`
		`  (a + b) / 2`
		`end`

Exemplo



```
fun media(a :: Number, b :: Number) -> Number:
  (a + b) / 2
end

media(2, 4)
```

Nome	Tipo	Valor (significado)
media	função	fun media(a, b): (a + b) / 2 end
a	constante	2
b	constante	4

Exemplo



```
fun media(a :: Number, b :: Number) -> Number:
  (a + b) / 2
end

media(2, 4)
>> 3
```

Nome	Tipo	Valor (significado)

media	função	`fun media(a, b):`
		`  (a + b) / 2`
		`end`

- As constantes a e b só existem durante a execução da função media!

